



课程设计（论文）

课程所属学院 信息与软件工程学院

所属课程名称 数据库原理及应用课程设计

项目 名称 考研自习室座位预约管理系统

院 （系） 计算机科学与技术（数据工程）系

班 级 计算机 2024-1 班

学 生 0102 侯凯飞

指 导 教 师 喻佳

辅 导 教 师 喻佳

2026 年 5 月 2 日



课程设计(论文)任务书

信工 学 院 计算机 2023 专 业 1 班

一、课程设计(论文)题目: 考研自习室座位预约管理系统

二、课程设计(论文)工作:

自 2026 年 4 月 15 日起至 2026 年 5 月 2 日止

三、课程设计(论文) 地点: 26-611、 图书馆

四、课程设计(论文)内容要求:

1. 本课程设计的目的

(1) 使学生掌握数据库开发的基本工作流程:能够运用基本科学原理,借助文献研究,确定本课程设计的选题,并给出对应选题的多种设计方案,通过讨论、分析确定设计的主要内容及需求分析。

(2) 巩固数据库课程所学的内容:熟练使用 SQL 语句,掌握构建数据库管理系统的六大过程并能够构建一个完整的数据库系统(包括数据库和数据库管理系统;

(3) 培养学生的计算机思维能力以及合作的精神:养成良好职业素养(如专业能力、管理决策能力以及团队协作能力等),具有自主学习意识和能力;

(4) 培养学生分析、解决问题的能力:整理、分析实验结果,运用数据库原理的相关理论完成设计可行性报告及课程设计论文(含需求分析、概要设计、详细设计等开发过程)等的撰写;

(5) 提高学生的科技论文写作能力。

2. 课程设计的任务及要求

1) 基本要求:

(1) 研究课程设计任务,并进行系统需求分析;

(2) 对系统进行总体设计,分解系统功能模块,进行任务分配,以实现分工合作;

- (3) 实现各功能模块代码;
- (4) 组装各模块, 并测试、完善系统。

2) 创新要求:

在基本要求达到后, 可进行创新设计, 如改进界面、增加功能或进行代码优化。

3) 课程设计论文编写要求

- (1) 要按照书稿的规格打印誊写课程设计论文
- (2) 论文包括封面、设计任务书(含评语)、摘要、目录、设计内容、设计小结
- (3) 论文装订按学校的统一要求完成

4) 参考文献:

- (1) 王珊. 数据库系统概论(第六版). 北京. 高等教育出版社, 2024
- (2) 夏邦国, 刘凡馨. Visual C++数据库开发经典实例精解. 机械工业出版社
- (3) <http://wenku.baidu.com/view/b083421ca76e58fafab00378.html>

.....

5) 课程设计进度安排

内容	天数	地点
系统总体设计	2	宿舍
软件设计及调试	4	实验室、图书馆
撰写报告	2	实验室、图书馆
答辩	2	实验室

学生签名: _____

2026 年 5 月 2 日

课程设计(论文)评审意见

(1) 课程设计过程 (20 分): 优 ()、良 ()、中 ()、一般 ()、差 ();

(2) 是否完成调试, 系统运行效果 (20 分): 优 ()、良 ()、中 ()、一般 ()、差 ();

(3) 回答问题 (30 分): 优 ()、良 ()、中 ()、一般 ()、差 ();

(4) 课程设计报告 (30 分): 优 ()、良 ()、中 ()、一般 ()、差 ();

(5) 格式规范性及考勤是否降等级: 是 ()、否 ()

评阅人: 喻佳 职称: 副教授

2026 年 5 月 10 日

考研自习室座位预约管理系统

摘要：

随着考研人数的逐年增加，学校自习室座位资源日益紧张，传统的“占座”模式容易引发纠纷且导致资源浪费。为了实现座位的科学分配与规范管理，本课程设计开发了一套考研自习室座位预约系统。该系统基于 B/S 架构，后端采用 Spring Boot 与 MyBatis-Plus 框架整合开发，前端采用 Vue3 结合 Element Plus 构建交互界面，并使用 MySQL 作为底层数据库。系统主要分为学生端与管理员端：学生登录后可以查看自习室状态、进行座位的分时段预约、签到及取消；管理员则负责物理资源（自习室、座位）的维护以及违约黑名单的管理。本系统通过严格的数据库事务控制和并发约束，有效防止了座位“超卖”与恶意占座现象，提高了自习室资源的利用率与管理效率。

目 录

1、项目内容、要求与分组情况总体介绍	2
2、概要介绍	3
2.1 开发工具及环境.....	3
2.2 任务分析.....	3
2.21 任务分析.....	3
2.22 系统设计目标.....	3
2.3 需求分析	4
2.31 功能要求	4
2.32 数据要求	5
2.33 数据字典	5
2.34 系统 E—R 图	8
2.35 数据库关系视图	9
2.36 系统设计流程图	9
2.4、功能模块设计	10
3、具体实现	11
3.1 类设计	11
3.2 模块及实现代码介绍(部分)	12
3.21、登录与权限控制模块.....	12
3.22、预约、取消与签到模块的具体实现.....	15
4 运行调试与分析讨论	18
4.1 用户登录的模块:	18
4.2 登录后的主界面对话框:	18
4.3. 学生预约与选座模块测试.....	19
4.4 管理员资源管理模块测试	20
4.5 黑名单与自动判违模块测试	21
5、设计体会与小结.....	25
6、参考文献	26

1、项目内容、要求与分组情况总体介绍

项目内容：

本课程设计的项目内容为设计并实现一个“考研自习室预约管理系统”。系统主要服务于有固定学习时间安排的学生群体，支持学生按日期、时间段和座位进行预约，并支持签到、取消、查看历史预约等操作；同时支持管理员维护自习室资源、管理预约记录、处理违规、管理黑名单以及维护学生账号。

与一般的简单座位预约系统相比，本系统更强调数据库建模与业务规则落地。例如：

- （1）系统以自习室、座位、预约、黑名单等为核心对象建立关系模型；
- （2）系统区分学生与管理员两类角色，并通过会话鉴权和角色拦截控制权限；
- （3）系统支持时间段预约与签到规则判断；
- （4）系统支持预约记录筛选、分页、自动违规处理等具有一定完整度的管理功能。

项目要求：

（1）安全性：严格的权限控制，对未授权人员他则无法做出相应操作，并且对在网络上传输的信息加密，防止信息被截获泄密或被非法修改。

（2）完整性：严格设计各表之间的联系，防止数据库中存在不符合语义的数据，也就是防止数据库中存在不正确的数据，当要删除的数据在被参照表中的时候，应该先删除参照表的数据；当增加或修改参照表中的数据的时候同时也应该相应的在被参照表中修改。以此确保数据库中包含的数据尽可能地准确和完整。

（3）项目可行性分析：

本项目为单人独立完成。负责涵盖需求分析、数据库概念与逻辑设计、前后端代码编写以及最终的系统联调与报告撰写。

2、概要介绍

2.1 开发工具及环境

操作系统：Windows 10 / 11

数据库管理系统：MySQL 8.0 及其可视化工具 Navicat

后端开发环境：JDK 17 + IntelliJ IDEA

后端技术栈：Spring Boot 3.2.5 + MyBatis-Plus

前端技术栈：Vue 3 + Element Plus + Axios

接口调试工具：Apifox

2.2 任务分析

2.2.1 任务分析

为了更深一步的学习数据库的知识，让自己可以把理论知识应用到实际的设计中去，增强自己的动手能力以及团队合作精神，我们以组为单位进行数据库的课程设计，我们选的是一个考研自习室座位预约管理系统的设计，具体所要完成的任务如下：

- 1) 结合实际情况进行分析，写出需求分析；
- 2) 根据需求分析设计出所需的界面；
- 3) 为界面和控件设计具体的功能实现函数；
- 4) 运行测试程序，调试并纠正运行的错误以及一些设计缺陷；
- 5) 调试无误，运行，并测试具体的项目功能。

2.2.2 系统设计目标

本系统的设计目标如下：

- (1) 实现学生与管理员双角色登录；
- (2) 实现自习室、座位、预约、黑名单等核心数据的增删改查；
- (3) 实现学生按日期和时间段预约座位；
- (4) 实现座位可视化展示，便于学生像选电影票一样选座；
- (5) 实现预约签到、取消预约、逾期自动判违；
- (6) 实现管理员对预约、黑名单和学生信息的维护；
- (7) 在保证功能完整性的同时，使系统界面清晰、业务逻辑尽量严谨。。

2.3 需求分析

2.3.1 功能要求

系统功能需求分为学生端和管理员端两部分。

一、学生端功能

- (1) 用户登录与退出；
- (2) 查看当前开放的自习室列表；
- (3) 选择日期和时间段后查看座位状态；
- (4) 通过可视化座位图选择座位；
- (5) 提交预约；
- (6) 取消尚未签到的预约；
- (7) 对当日预约进行签到；
- (8) 查看我的预约记录；
- (9) 查看当前黑名单状态；
- (10) 查看个人违规记录。

二、管理员端功能

- (1) 新增、修改、启用/停用、删除自习室；
- (2) 按房间管理座位；
- (3) 在新增自习室时批量自动生成座位，减少手工录入工作量；
- (4) 查看预约记录，并按学号、自习室、日期、时间段、状态筛选；
- (5) 对预约记录进行分页展示；
- (6) 手动取消预约或手动标记违约；
- (7) 扫描逾期未签到预约并自动处理；
- (8) 新增黑名单记录、解除黑名单；
- (9) 对黑名单记录进行分页展示；
- (10) 新增、修改、删除学生账号；
- (11) 删除学生时，若存在未来预约，需二次确认并自动释放相应座位资源。

三、业务规则要求

- (1) 同一学生每天在上午、下午、晚上三个时间段中，每个时间段最多保留一条有效预约；
- (2) 同一时间段内，同一座位只能被一人有效预约；
- (3) 学生不能预约过去的日期；
- (4) 若当天某时间段已开始，则不能再预约当天该时间段；
- (5) 上午签到时段为 8:00-12:00，超过 8:30 未签到视为违规；
- (6) 下午签到时段为 14:00-17:00，超过 14:30 未签到视为违规；
- (7) 晚上签到时段为 18:00-22:00，超过 18:30 未签到视为违规；
- (8) 被黑名单封禁的学生不能继续预约；
- (9) 取消某时间段预约后，该学生可以重新预约该时间段的其他座位。

2.32 数据要求

本系统主要涉及五类核心数据：

- (1) 用户数据：保存学生和管理员的登录身份、姓名、角色等信息；
- (2) 自习室数据：保存房间名称、楼层、总座位数、开放状态等；
- (3) 座位数据：保存座位所属房间、座位编号、是否靠窗、是否有插座、是否损坏等；
- (4) 预约数据：保存学生、座位、日期、时间段、预约状态、创建时间等；
- (5) 黑名单数据：保存学生学号、违规原因、违规时间、封禁状态等。

其中预约数据是整个系统的核心业务数据，它与学生、座位、自习室以及黑名单逻辑均有联系，因此在设计时需要重点考虑其完整性、一致性和状态流转。

2.33 数据字典

1. 数据项：

- 1) 学号：唯一标识学生的代号，别名：student_id，字符型 (varchar)；
- 2) 密码：用户的登录凭证，别名：password，字符型 (varchar)；
- 3) 真实姓名：学生的姓名，别名：real_name，字符型 (varchar)；
- 4) 角色：标识普通学生或管理员，别名：role，整型 (int)；
- 5) 自习室名称：标识自习室，别名：room_name，字符型 (varchar)；
- 6) 座位编号：标识具体的座位，别名：seat_number，字符型 (varchar)；
- 7) 预约日期：指明预约的具体日期，别名：reservation_date，日期型 (date)；
- 8) 时间段：指明上午/下午/晚上，别名：time_slot，整型 (tint)；
- 9) 预约状态：指明该条记录的进度（如已签到），别名：status，整型 (int)；
- 10) 违规原因：记录为何被拉入黑名单，别名：reason，字符型 (varchar)。

2. 数据结构：

- 1) 用户：User，组成：ID + 学号 + 密码 + 真实姓名 + 角色 + 创建时间。
- 2) 自习室：Room，组成：ID + 自习室名 + 楼层 + 总座位数 + 状态。
- 3) 座位：Seat，组成：ID + 自习室 ID + 座位编号 + 靠窗标记 + 插座标记 + 状态。
- 4) 预约流水：Reservation，组成：ID + 学号 + 座位 ID + 预约日期 + 时间段 + 状态 + 创建时间。
- 5) 违规记录：Blacklist，组成：ID + 学号 + 违规原因 + 违规时间 + 状态。

3. 数据流词条描述：

(1) 登录验证描述

数据流	登录验证
数据流来源	学生/管理员客户端页面输入
数据去向	身份验证处理模块 -> user 信息表
数据流组成	学号、密码、角色类型

(2) 自习室与座位查询描述

数据流	自习室与座位查询
数据流来源	数据库中的 room 和 seat 表
数据去向	预约系统选座展示页面
数据流组成	自习室 ID、自习室名称、总座位数、座位编号、靠窗/插座标记、物理状态

(3) 座位预约请求描述

数据流	座位预约请求
数据流来源	选座页面（学生提交预约申请）
数据去向	预约核心控制模块 -> reservation 预约流水表
数据流组成	学号、座位 ID、预约日期、时间段（上午/下午/晚上）

（4）签到与状态更新描述

数据流	签到与状态更新
数据流来源	学生个人预约记录页面（点击签到或取消）
数据去向	状态更新模块 -> reservation 预约流水表
数据流组成	预约流水主键 ID、操作人学号、更新后的状态（1-已签到 / 2-已取消）

（5）违规惩罚描述

数据流	黑名单判定
数据流来源	违约判定模块（系统后台超时未签到判定）
数据去向	blacklist 违规黑名单表
数据流组成	违规人学号、违规原因、违规日期、封禁状态

4.数据存储

1. 名字：用户信息

描述：自习室座位预约系统中存储的所有用户（包含普通学生和管理员）基础信息。

定义：用户信息 = 主键 ID + 学号 + 密码 + 真实姓名 + 角色 + 创建时间

位置：存储在 user 表中。输出供登录身份验证、权限判定和个人信息查询使用。

2. 名字：自习室信息

描述：系统中管理的自习室基础物理资源信息。

定义：自习室信息 = 自习室 ID + 自习室名称 + 所在楼层 + 总座位数 + 状态（开放/关闭）

位置：存储在 room 表中。输出供学生端自习室列表展示及座位查询使用。

3. 名字：座位信息

描述：每个自习室内的具体座位详情及其硬件设备状态信息。

定义：座位信息 = 座位 ID + 所属自习室 ID + 座位编号 + 是否靠窗 + 是否有插座 + 状态（正常/损坏）

位置：存储在 seat 表中。输出供选座网格渲染和预约逻辑匹配使用。

4. 名字：预约记录信息

描述：学生发起座位预约后产生的核心业务流水数据。

定义：预约记录信息 = 预约记录 ID + 预约人学号 + 预约座位 ID + 预约日期 + 时间段（上午/下午/晚上） + 预约状态（已预约/已签到/已取消/违约） + 提交时间

位置：存储在 reservation 表中。输出供学生历史记录查询、入座签到、以及系统防并发冲突校验使用。

5. 名字：违规黑名单信息

描述：记录未按规定时间签到的学生违约情况及当前封禁状态。

定义：违规黑名单信息 = 违规记录 ID + 违规人学号 + 违规原因 + 违规日期 + 封禁状态（生效中/已解除）

位置：存储在 blacklist 表中。输出供预约权限拦截及学生个人信誉中心查询使用。

5.处理过程

1. 名字：登录身份验证处理

描述：用户登录系统时的身份合法性及权限校验。

输入：学号，密码

输出：匹配成功返回相应用户信息及角色（0-学生，1-管理员）；匹配失败返回错误提示。

2. 名字：资源与状态查询处理

描述：学生进入选座页面前，获取自习室及对应座位的占用情况。

输入：自习室 ID，查询日期，查询时间段

输出：返回指定条件下的所有座位列表，并结合预约表（reservation）过滤出各座位的当前状态（可选/已被占/维修中）。

3. 名字：座位预约业务处理

描述：核心的并发预约逻辑处理。系统需依次拦截黑名单用户、校验每人每日限额（防重复）、校验座位排他性（防超卖）。

输入：学号，座位 ID，预约日期，时间段

输出：若各项校验通过，则新增一条记录至 reservation 表，状态设为“0-已预约”，返回预约成功功能；若校验未通过，输出具体的拦截原因。

4. 名字：签到与取消处理

描述：更新已存在的预约流水的业务状态。

输入：预约记录 ID，学号，操作类型指示（签到/取消）

输出：在 reservation 表中将对应记录的状态更新为“1-已签到”或“2-已取消”，并释放相关座位资源状态。

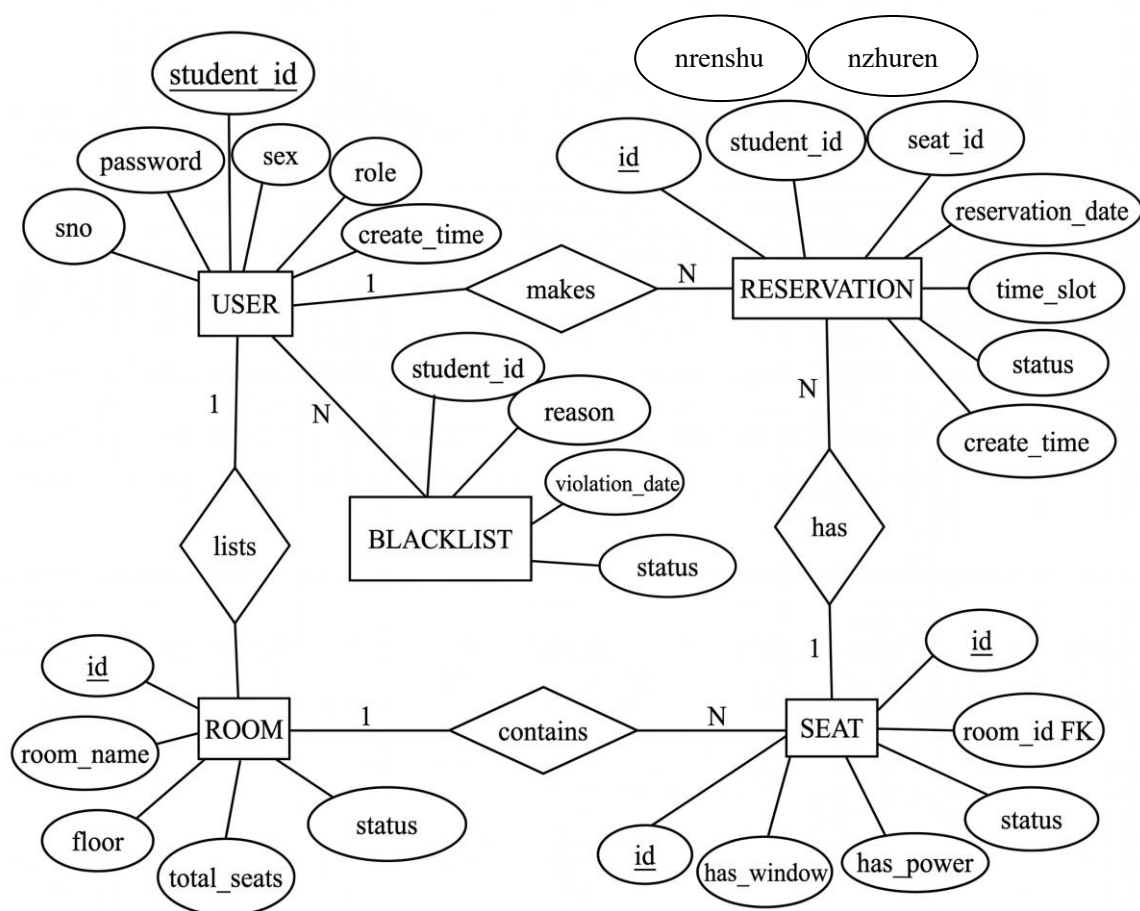
5. 名字：违约与黑名单判定处理

描述：处理学生逾期未签到的违规行为。

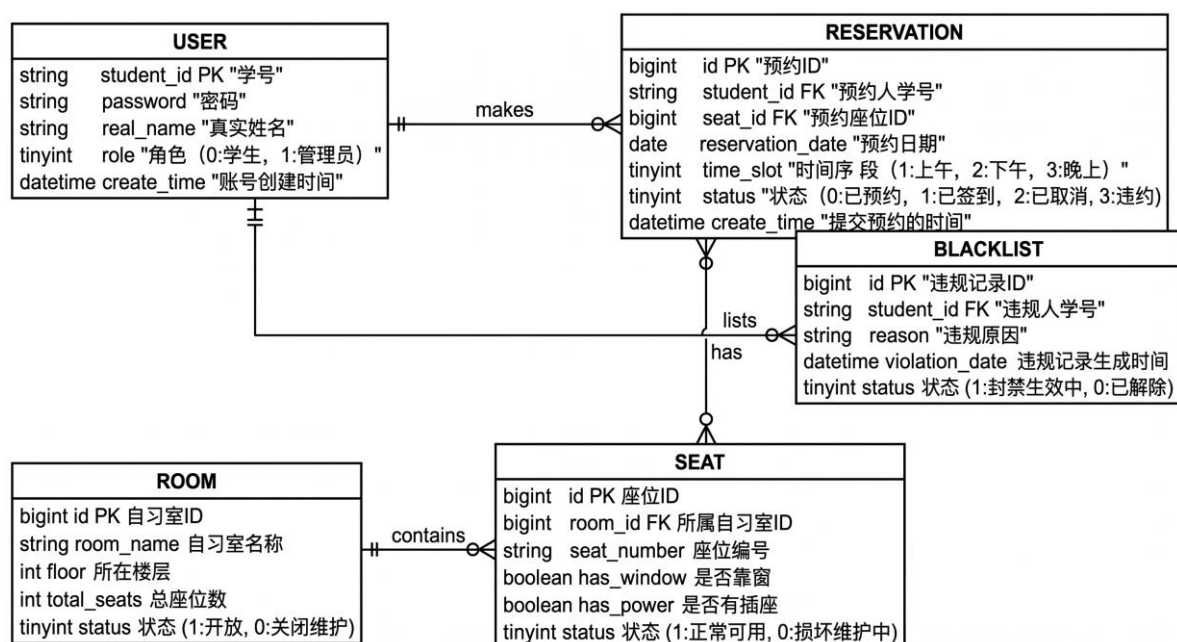
输入：未在规定时间内签到的预约流水记录（包含学号与违规详情）

输出：将该预约流水状态更新为“3-违约”，并向 blacklist 表中自动插入一条新记录，封禁状态设为“1-生效中”，限制该学号后续的预约操作。成

2.34 系统 E—R 图

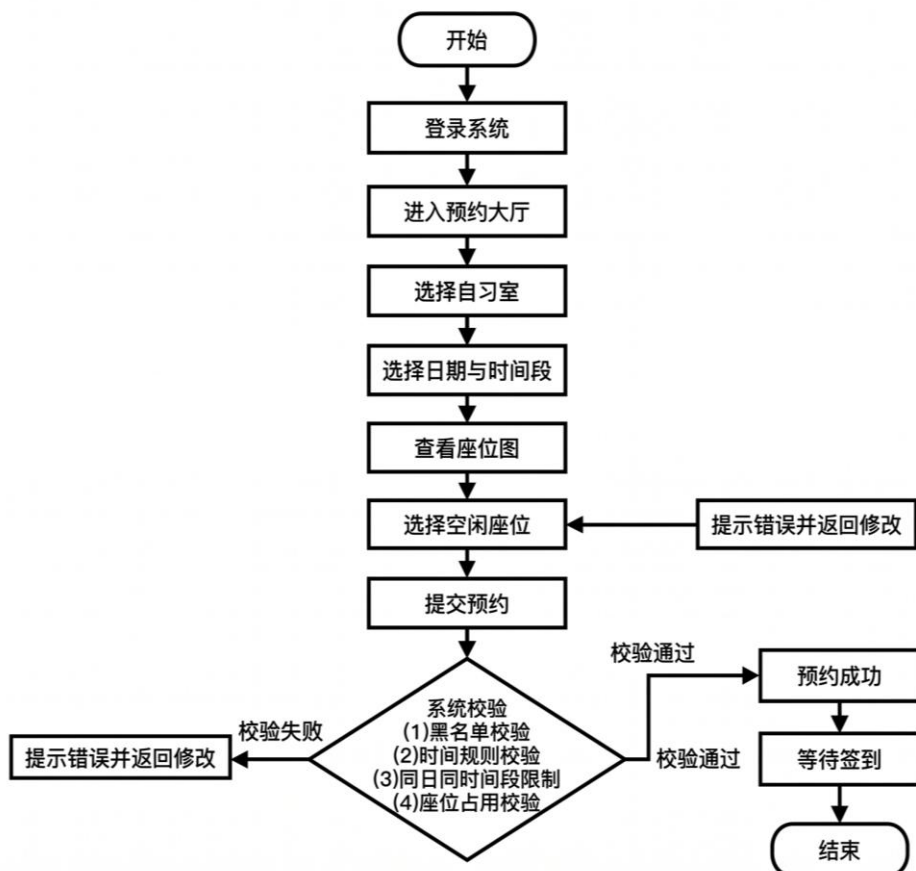


2.35 数据库关系视图

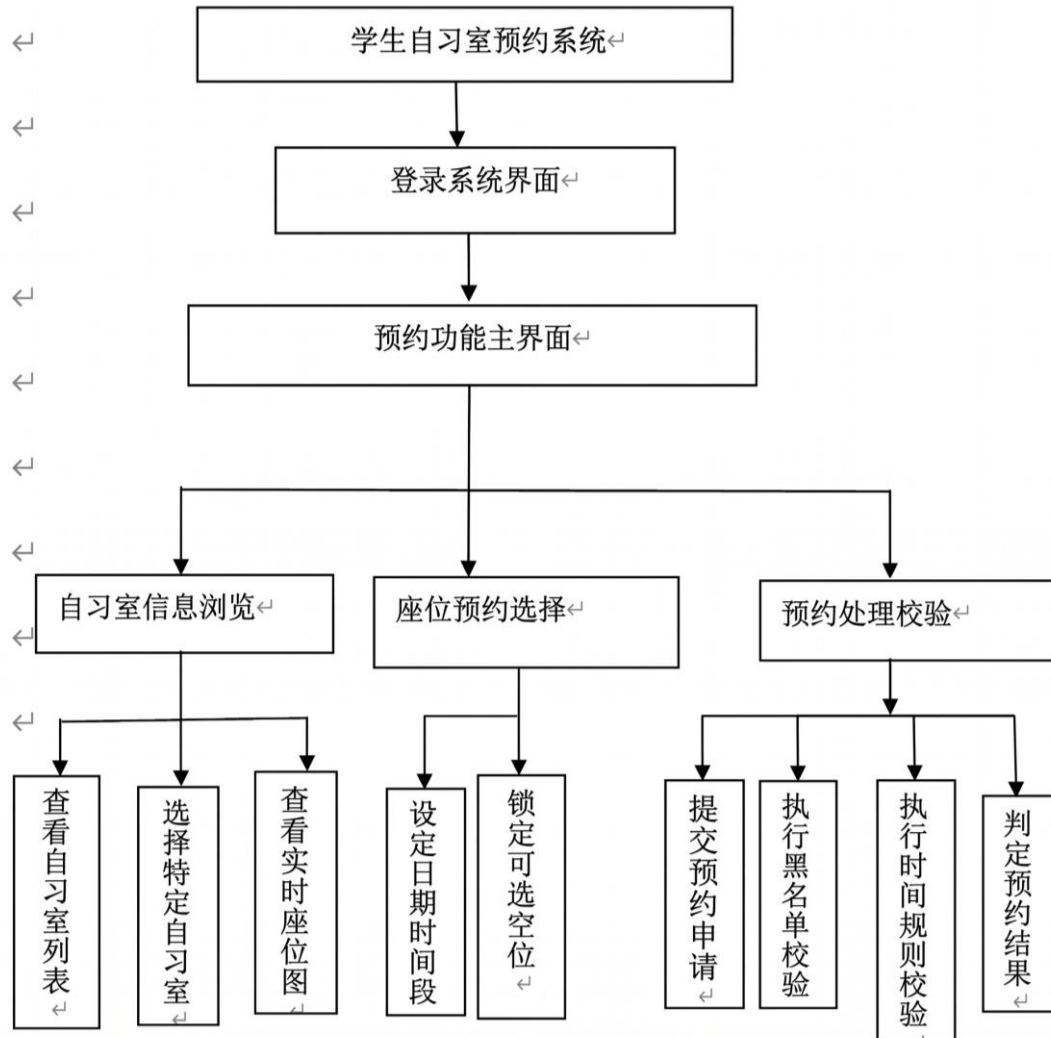


2.36 系统设计流程图

1. 登录流程图:



2. 功能流图



2.4、功能模块设计

(1) 用户与鉴权模块

负责登录、退出、获取当前会话用户信息，并区分学生与管理员权限。

(2) 学生预约模块

负责学生查看开放自习室、查看座位状态、选择座位、提交预约、取消预约、签到、查看预约记录。

(3) 信誉与黑名单模块

负责查询黑名单状态、违规记录、封禁状态控制、手动或自动加入黑名单、解除黑名单。

(4) 资源管理模块

负责管理员对自习室和座位资源进行维护，包括新增房间、自动生成座位、修改座位属性、删除房间与座位。

(5) 预约管理模块

负责管理员对预约记录进行筛选、分页、取消预约、标记违约、扫描逾期预约。

(6) 学生管理模块

负责管理员新增、修改、删除学生账号，并在删除时联动处理未来预约。

3、具体实现

3.1 数据库访问与事务控制设计

本系统在编码实现阶段，核心围绕对底层 MySQL 关系型数据库的数据访问、复杂关联查询以及多并发环境下的事务控制展开。

一、 数据访问与映射层设计（ORM 映射）

系统采用对象关系映射技术（MyBatis-Plus 框架），将建立的 5 张关系数据表封装为可以直接进行 CRUD 操作的数据访问对象（DAO/Mapper）。

基础资源访问：针对 room 和 seat 表，系统底层通过外键关联（room_id）实现对“自习室-座位”一对多结构的完整读取，保障物理资源的展示一致性。

鉴权与身份访问：针对 user 表，系统实现基于唯一键（学号 student_id）的高效索引查询，用于核对密码并提取 role 字段完成角色赋权。

业务流水访问：针对高频读写的 reservation 和 blacklist 表，构建了丰富的条件构造器（Wrapper），支持按照时间段、日期区间、学生学号等多维度字段进行复合查询与数据过滤。

二、 核心事务与并发控制机制

在系统最核心的“座位预约”处理流程中，存在严格的排他性要求。为防止高并发情况下的“座位超卖”或“重复占座”，系统在处理逻辑中严格启用了数据库事务（Transaction），确保多步 SQL 操作的原子性（ACID）。其底层执行的数据库校验逻辑如下：

信誉拦截：执行 SELECT 语句联合 blacklist 表，核实当前操作者是否处于封禁期。

限额校验：利用聚合函数 COUNT() 对 reservation 表进行查询，判断该学生当日有效预约数是否已达上限。

排他性冲突检测：通过查询 WHERE seat_id = ? AND reservation_date = ? AND time_slot = ? AND status IN (0,1)，检索目标座位在特定时段是否已被他人占用。

事务提交与回滚：当且仅当以上所有前置条件查询均通过后，系统才执行 INSERT 语句将预约记录写入数据库。若任意校验失败或网络异常，系统触发事务回滚，保障数据库状态的绝对一致。

三、 多表关联与复杂查询设计

在实现“查看我的预约”或管理员界面的“预约大厅”时，单张流水表的数据无法满足视图需求。系统在逻辑执行时，通过多表关联或分步 ID 映射的方式，提取 reservation 表中的 seat_id 去匹配 seat 表的属性（如座位编号），再提取 room_id 去匹配 room 表的属性（如自习室名称）。这种数据拼装逻辑充分利用了数据库的主外键参照完整性，向前端输出包含完整物理位置和业务流水的综合数据集合。

四、 数据库状态机流转设计

系统中的核心业务数据并非静态存储，而是随着业务操作通过 UPDATE 语句进行状态机流转：

签到与取消逻辑：通过定向更新 reservation 表中的 status 字段，实现从“0-已预约”向“1-已签到”或“2-已取消”的单向状态更新，并即时释放或确认对应的座位使用权。

自动判违逻辑：系统底层结合定时任务，定期执行批量扫描（SELECT ... WHERE status = 0 AND 当前时间 > 签到截止时间），将符合条件的记录批量 UPDATE 为“3-违约”，并立即触发对 blacklist 表的 INSERT 操作，实现信誉数据的闭环记录。

3.2 模块及实现代码介绍(部分)

3.21、登录与权限控制模块

```
@RestController
@RequestMapping("/api/user")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping("/login")
    public Result<User> login(@Valid @RequestBody LoginRequest request, HttpSession session) {
        User user = userService.login(request.getStudentId().trim(), request.getPassword());
        if (user != null) {
            session.setAttribute(AuthConstants.LOGIN_USER,
                                new LoginUser(user.getId(), user.getStudentId(), user.getRealName(),
user.getRole()));
            return Result.success("登录成功", user);
        }
        return Result.error("学号或密码错误");
    }

    @GetMapping("/me")
    public Result<LoginUser> me(HttpSession session) {
        return Result.success(AuthHelper.requireLoginUser(session));
    }

    @PostMapping("/logout")
    public Result<String> logout(HttpSession session) {
        session.invalidate();
        return Result.success("退出登录成功", null);
    }
}
```

注：系统采用会话方式进行身份认证。登录时，后端根据学号和密码查询用户表，若成功则在 `HttpSession` 中写入当前登录用户信息。随后由拦截器对需要登录的接口进行保护，并对管理员接口额外进行角色校验。

前端路由部分根据当前用户角色进行分流：

- (1) 学生登录后进入学生端；
- (2) 管理员登录后进入管理员端；
- (3) 未登录用户访问受限页面时会被重定向到登录页。

```

<script setup>
import { reactive, ref } from 'vue'
import { useRouter } from 'vue-router'
import { useSessionStore } from '../stores/session'

const router = useRouter()
const session = useSessionStore()

const form = reactive({
  studentId: "",
  password: "",
})

const errorMessage = ref("")
const loading = ref(false)

async function handleLogin() {
  errorMessage.value = ""
  loading.value = true
  try {
    const user = await session.signIn({
      studentId: form.studentId.trim(),
      password: form.password,
    })
    await router.push(user.role === 1 ? '/admin' : '/student')
  } catch (error) {
    errorMessage.value = error.message || '登录失败'
  } finally {
    loading.value = false
  }
}

function fillDemo(role) {
  if (role === 'admin') {
    form.studentId = 'ADMIN01'
    form.password = 'admin123'
    return
  }
  form.studentId = 'STU2001'
  form.password = 'pass123'
}
</script>

<template>
<div class="login-shell">
  <section class="login-card">

```

```

<div class="login-brand">
  <p class="eyebrow">SRS Frontend</p>
  <h1>考研自习室预约管理系统</h1>
  <p class="subtitle">
    基于 Vue 3 的前后端分离前端。当前接入 Spring Boot 会话鉴权接口，支持学生端
    与管理员端分流。
  </p>
</div>

<form class="login-form" @submit.prevent="handleLogin">
  <label>
    <span>学号</span>
    <input v-model="form.studentId" placeholder="请输入学号" autocomplete="username"
  />
  </label>

  <label>
    <span>密码</span>
    <input
      v-model="form.password"
      type="password"
      placeholder="请输入密码"
      autocomplete="current-password"
    />
  </label>

  <p v-if="errorMessage" class="form-error">{{ errorMessage }}</p>

  <button class="primary-btn" type="submit" :disabled="loading">
    {{ loading ? '登录中...' : '登录系统' }}
  </button>
</form>

<div class="demo-panel">
  <p>演示账号</p>
  <div class="demo-actions">
    <button type="button" class="ghost-btn" @click="fillDemo('student')">
      填入学生账号
    </button>
    <button type="button" class="ghost-btn" @click="fillDemo('admin')">
      填入管理员账号
    </button>
  </div>
</div>
</section>
</div>

```

</template>

3. 22、预约、取消与签到模块的具体实现

```
@RestController
@RequestMapping("/api/reservation")
public class ReservationController {

    private final ReservationService reservationService;
    private final SeatService seatService;
    private final RoomService roomService;

    public ReservationController(
        ReservationService reservationService,
        SeatService seatService,
        RoomService roomService) {
        this.reservationService = reservationService;
        this.seatService = seatService;
        this.roomService = roomService;
    }

    @PostMapping("/reserve")
    public Result<String> reserve(
        @RequestParam Long seatId,
        @RequestParam @DateTimeFormat(pattern = "yyyy-MM-dd") LocalDate date,
        @RequestParam Integer timeSlot,
        HttpSession session) {

        LoginUser loginUser = AuthHelper.requireLoginUser(session);
        String message = reservationService.reserveSeat(loginUser.getStudentId(), seatId, date,
timeSlot);
        if ("success".equals(message)) {
            return Result.success("预约成功，请在规定签到时间内完成签到。", null);
        }
        return Result.error(message);
    }

    @PostMapping("/checkIn")
    public Result<String> checkIn(@RequestParam Long reservationId, HttpSession session) {
        LoginUser loginUser = AuthHelper.requireLoginUser(session);
        String message = reservationService.checkIn(reservationId, loginUser.getStudentId());
        if ("success".equals(message)) {
            return Result.success("签到成功，祝你学习顺利。", null);
        }
        return Result.error(message);
    }
}
```

```

    }

    @PostMapping("/cancel")
    public Result<String> cancel(@RequestParam Long reservationId, HttpSession session) {
        LoginUser loginUser = AuthHelper.requireLoginUser(session);
        boolean success = reservationService.cancelReservation(reservationId,
loginUser.getId());
        if (success) {
            return Result.success("已成功取消预约，座位已释放。", null);
        }
        return Result.error("取消失败");
    }

    @GetMapping("/my")
    public Result<List<Reservation>> myReservations(
        @RequestParam(required = false) @DateTimeFormat(pattern = "yyyy-MM-dd")
LocalDate date,
        @RequestParam(required = false) Integer status,
        HttpSession session) {
        LoginUser loginUser = AuthHelper.requireLoginUser(session);
        List<Reservation> reservations = reservationService.lambdaQuery()
            .eq(Reservation::getStudentId, loginUser.getId())
            .eq(date != null, Reservation::getReservationDate, date)
            .eq(status != null, Reservation::getStatus, status)
            .orderByDesc(Reservation::getReservationDate)
            .orderByDesc(Reservation::getCreateTime)
            .list();
        enrichReservationDisplay(reservations);
        return Result.success(reservations);
    }

    private void enrichReservationDisplay(List<Reservation> reservations) {
        if (reservations == null || reservations.isEmpty()) {
            return;
        }

        Set<Long> seatIds = reservations.stream()
            .map(Reservation::getSeatId)
            .filter(id -> id != null)
            .collect(Collectors.toSet());
        Map<Long, Seat> seatMap = seatService.listByIds(seatIds).stream()
            .collect(Collectors.toMap(Seat::getId, Function.identity()));

        Set<Long> roomIds = seatMap.values().stream()
            .map(Seat::getRoomId)
            .filter(id -> id != null)

```

```

        .collect(Collectors.toSet());
    Map<Long, Room> roomMap = roomService.listByIds(roomIds).stream()
        .collect(Collectors.toMap(Room::getId, Function.identity()));

    for (Reservation reservation : reservations) {
        Seat seat = seatMap.get(reservation.getSeatId());
        if (seat == null) {
            continue;
        }

        reservation.setSeatNumber(seat.getSeatNumber());
        Room room = roomMap.get(seat.getRoomId());
        if (room != null) {
            reservation.setRoomName(room.getRoomName());
        }
    }
}

```

注：预约模块是系统的核心业务模块，实现时主要考虑了以下规则：

- （1）黑名单用户不能预约；
- （2）同一天的同一时间段，学生只能保留一条有效预约；
- （3）同一时间段同一座位只能被一人占用；
- （4）当天若某时段已开始，则不能再预约当天该时段；
- （5）学生取消某一时段预约后，可重新预约该时段的座位。

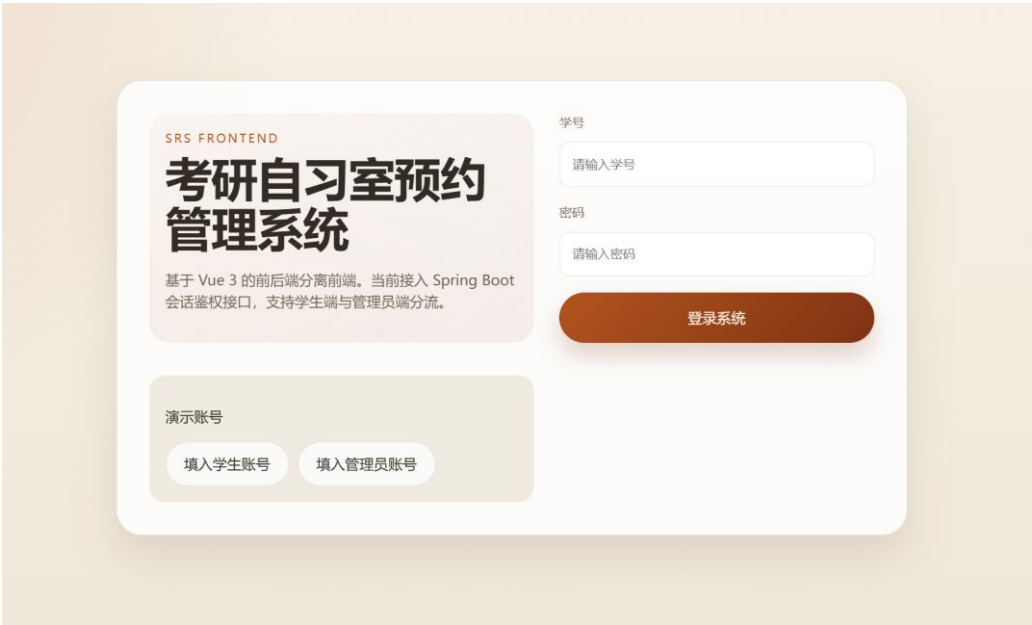
为避免并发下出现重复预约，后端在 `ReservationServiceImpl` 中为“用户-日期-时间段”和“座位-日期-时间段”分别建立锁键，在预约时按排序后的锁键加锁，降低并发冲突概率。

签到模块按时间段规则工作：

- （1）上午可在 8:00 后签到，8:30 后未签到判违约；
- （2）下午可在 14:00 后签到，14:30 后未签到判违约；
- （3）晚上可在 18:00 后签到，18:30 后未签到判违约。

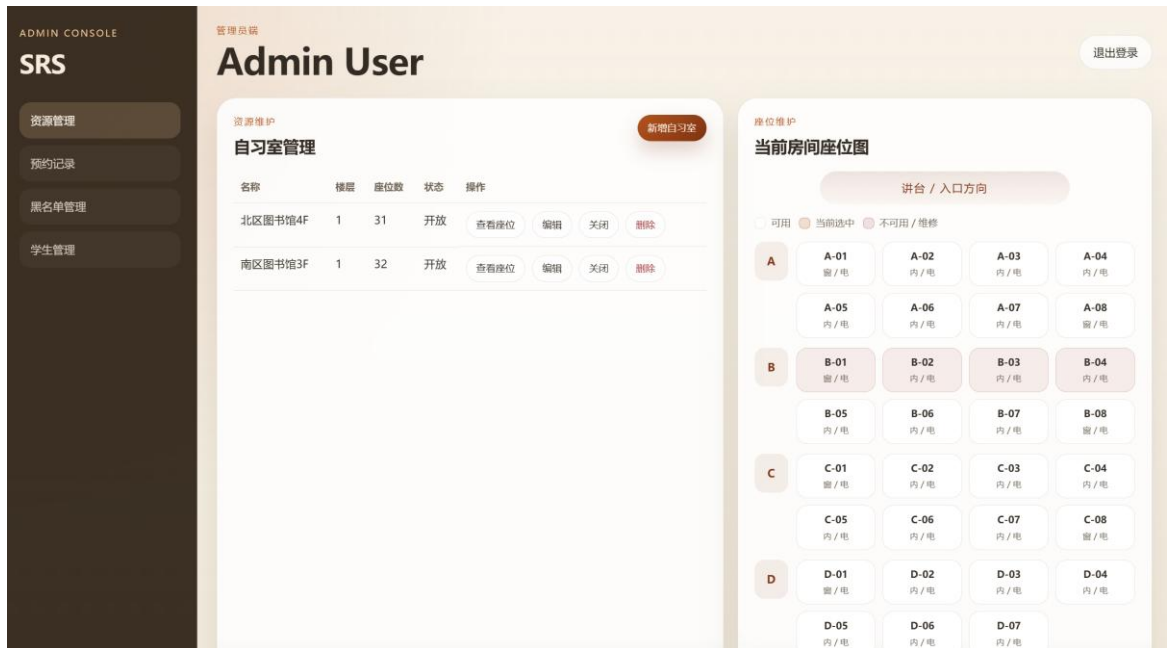
4 运行调试与分析讨论

4.1 用户登录的模块：



4.2 登录后的主界面对话框：





4.3. 学生预约与选座模块测试

测试内容：

- (1) 查看开放自习室；
- (2) 切换日期和时间段刷新座位状态；
- (3) 选择空闲座位提交预约；
- (4) 预约后再次查看座位图，已预约座位是否变红；
- (5) 取消预约后是否可以重新预约；
- (6) 能否预约当天已开始的时间段。

测试结果：

- (1) 系统能正确展示房间和座位；
- (2) 已预约座位会被标记为不可用；
- (3) 同一学生同一时间段不能重复保留有效预约；
- (4) 取消后可重新预约；
- (5) 若当前时间已过某时间段开始时间，则不能再预约当天该时段；
- (6) 整体流程符合设计预期。

可用

当前选中

不可用 / 维修

A

A-01
窗 / 电

A-02
内 / 电

A-03
内 / 电

A-04
内 / 电

A-05
内 / 电

A-06
内 / 电

A-07
内 / 电

A-08
窗 / 电

B

B-01
窗 / 电

B-02
内 / 电

B-03
内 / 电

B-04
内 / 电

B-05
内 / 电

B-06
内 / 电

B-07
内 / 电

B-08
窗 / 电

C

C-01
窗 / 电

C-02
内 / 电

C-03
内 / 电

C-04
内 / 电

C-05
内 / 电

C-06
内 / 电

C-07
内 / 电

C-08
窗 / 电

D

D-01
窗 / 电

D-02
内 / 电

D-03
内 / 电

D-04
内 / 电

D-05
内 / 电

D-06
内 / 电

D-07
内 / 电

红色座位表示该日期该时段已被预约或当前不可用，无法再次选择。

已选座位: A-03

步骤 2

提交预约

预约日期

2026/04/27

时间段

下午 (14:00-17:00, 14:30前签到)

切换日期或时间段后，系统会自动刷新当前已被预约的座位状态。

预约成功

确认预约

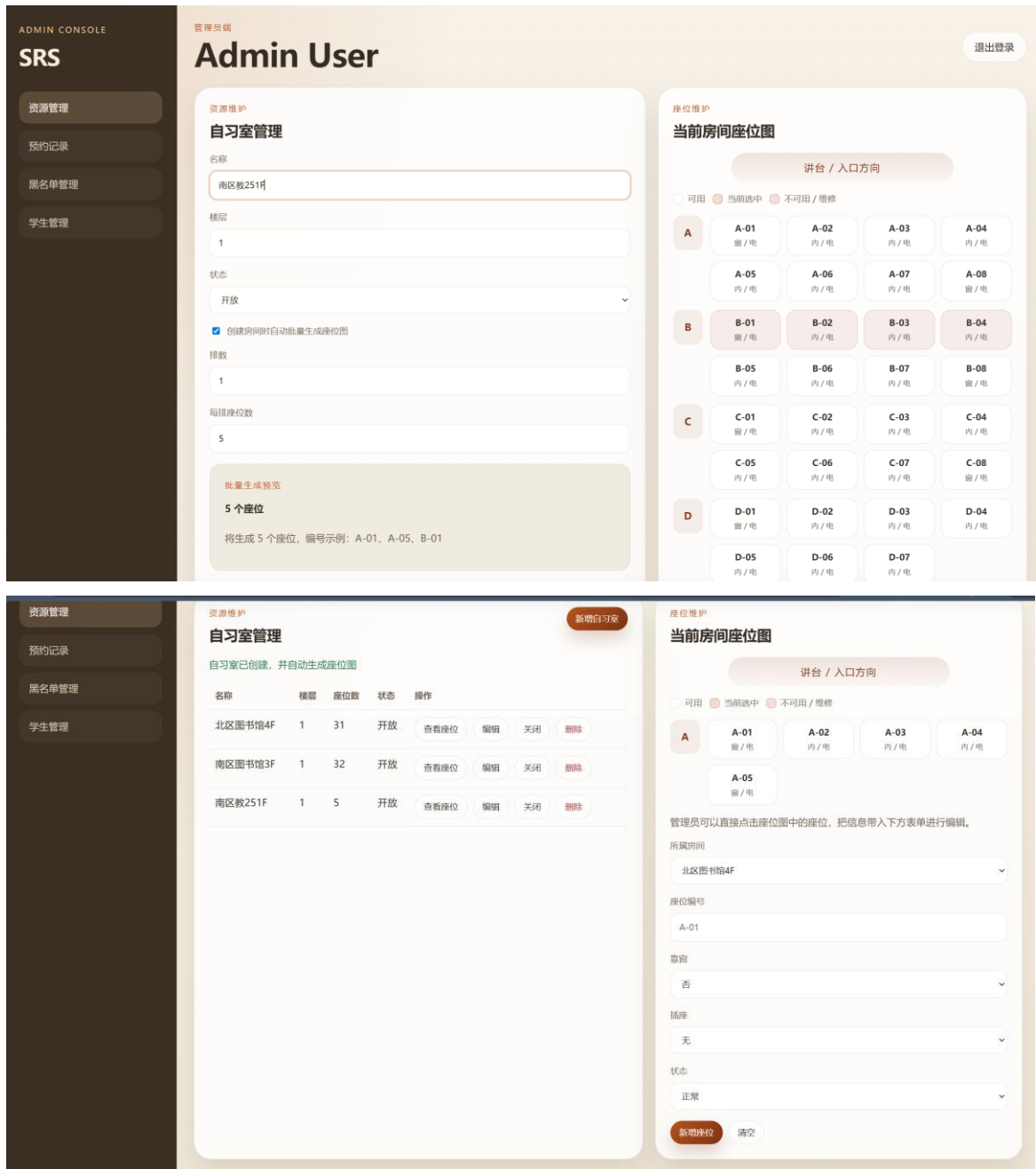
4.4 管理员资源管理模块测试

测试内容：

- （1）新增自习室；
- （2）使用自动生成座位功能批量创建座位；
- （3）修改自习室信息；
- （4）删除带有座位的自习室；
- （5）停用与启用自习室状态。

测试结果：

- （1）新增自习室成功；
- （2）批量生成座位成功，编号规则正确；
- （3）修改房间信息后能即时生效；
- （4）删除自习室时可连同其下座位一起删除；
- （5）停用后学生端不再展示该房间，启用后恢复展示。



4.5 黑名单与自动判违模块测试

测试内容：

- (1) 学生预约后故意不签到；
- (2) 管理员执行逾期扫描；
- (3) 等待定时任务自动扫描；
- (4) 检查预约状态是否改为违约；
- (5) 检查黑名单表是否新增记录；
- (6) 检查学生端是否被禁止继续预约；
- (7) 管理员解除黑名单后是否可恢复预约。

测试结果：

- (1) 超过签到截止时间的待签到预约会被正确判为违约；
- (2) 系统会自动写入黑名单记录；
- (3) 被封禁的学生无法再次预约；
- (4) 解除封禁后学生可恢复正常预约；
- (5) 说明自动判违链路完整可用。

ADMIN CONSOLE

SRS

资源管理

预约记录

黑名单管理

学生管理

管理员端

Admin User

退出登录

BLACKLIST ADMIN

黑名单管理

支持手动新增黑名单、解除封禁，并按学号与状态筛选记录，列表每页展示 10 条。

学号

例如 STU2001

封禁原因

请输入封禁原因

新增黑名单

学号

输入学号筛选

状态

全部

查询

ID	学号	原因	违规时间	状态	操作
12	STU1002	2026-04-27 上午预约未签到	2026-04-27T08:30:46	封禁中	解除封禁
11	STU1001	2026-04-27 上午预约未签到	2026-04-27T08:30:46	封禁中	解除封禁
10	STU1001	2026-04-26 上午预约未签到	2026-04-26T19:19:26	已解除	

共 3 条，当前第 1 / 1 页

上一页

下一页

STUDENT CONSOLE

SRS

预约大厅

我的预约

我的信誉

学生端

你好，张三

退出登录

预约记录

我的预约

当前签到规则：上午 8:30 前、下午 14:30 前、晚上 18:30 前完成签到；页面每 30 秒自动刷新签到状态提示。

日期筛选

yyyy/mm/日

状态筛选

全部

查询

ID	自习室	座位号	日期	时段	状态	签到提示	操作
38	北区图书馆4F	A-02	2026-04-27	晚上 (18:00-22:00)	待签到	未到签到时间，可于 18:00 后签到	签到取消
37	北区图书馆4F	A-01	2026-04-27	下午 (14:00-17:00)	已取消	当前记录无需签到	
36	北区图书馆4F	A-01	2026-04-27	上午 (8:00-12:00)	已违约	当前记录无需签到	
33	北区图书馆4F	A-01	2026-04-26	晚上 (18:00-22:00)	已违约	当前记录无需签到	
32	北区图书馆4F	A-02	2026-04-26	下午 (14:00-17:00)	已违约	当前记录无需签到	
31	北区图书馆4F	A-01	2026-04-26	上午 (8:00-12:00)	已违约	当前记录无需签到	

C

C-01
窗 / 电

C-02
内 / 电

C-03
内 / 电

C-04
内 / 电

C-05
内 / 电

C-06
内 / 电

C-07
内 / 电

D

D-01
窗 / 电

D-02
内 / 电

D-03
内 / 电

D-04
内 / 电

D-05
内 / 电

D-06
内 / 电

D-07
内 / 电

红色座位表示该日期该时段已被预约或当前不可用，无法再次选择。

已选座位：D-02

步骤 2

提交预约

预约日期

2026/04/27

时间段

晚上（18:00-22:00，18:30前签到）

切换日期或时间段后，系统会自动刷新当前已被预约的座位状态。

黑名单状态不可预约

管理员端

Admin User

退出登录

BLACKLIST ADMIN

黑名单管理

支持手动新增黑名单、解除封禁，并按学号与状态筛选记录，列表每页展示 10 条。

学号

例如 STU2001

封禁原因

请输入封禁原因

新增黑名单

学号

输入学号筛选

状态

全部

查询

ID	学号	原因	违规时间	状态	操作
12	STU1002	2026-04-27 上午预约未签到	2026-04-27T08:30:46	封禁中	解除封禁
11	STU1001	2026-04-27 上午预约未签到	2026-04-27T08:30:46	已解除	
10	STU1001	2026-04-26 上午预约未签到	2026-04-26T19:19:26	已解除	

共 3 条，当前第 1 / 1 页

上一页

下一页

A

A-01
窗 / 电

A-02
内 / 电

A-03
内 / 电

A-04
内 / 电

A-05
内 / 电

A-06
内 / 电

A-07
内 / 电

A-08
窗 / 电

B

B-01
窗 / 电

B-02
内 / 电

B-03
内 / 电

B-04
内 / 电

B-05
内 / 电

B-06
内 / 电

B-07
内 / 电

B-08
窗 / 电

C

C-01
窗 / 电

C-02
内 / 电

C-03
内 / 电

C-04
内 / 电

C-05
内 / 电

C-06
内 / 电

C-07
内 / 电

C-08
窗 / 电

D

D-01
窗 / 电

D-02
内 / 电

D-03
内 / 电

D-04
内 / 电

D-05
内 / 电

D-06
内 / 电

D-07
内 / 电

D-08
窗 / 电

红色座位表示该日期该时段已被预约或当前不可用，无法再次选择。

已选座位: A-02

步骤 2

提交预约

预约日期

2026/04/28

时间段

晚上 (18:00-22:00, 18:30前签到)

切换日期或时间段后，系统会自动刷新当前已被预约的座位状态。

预约成功

确认预约

24

5、设计体会与小结

通过本次课程设计，我对数据库系统从需求分析到落地实现的全过程有了更完整的理解。以前在课堂中更多接触的是数据库表、SQL 语句和范式等静态知识，而在实际项目中，我需要把这些知识转化成真正可运行的系统，这使我更加理解“数据库设计不仅是建表，更是业务规则建模”的含义。

在本项目中，我最大的收获主要体现在以下几个方面：

- (1) 进一步掌握了数据库建模方法，能够围绕业务对象抽取实体并建立联系；
- (2) 学会了将业务规则映射为数据库查询条件和状态流转逻辑；
- (3) 理解了前后端分离项目中接口设计、权限控制和页面交互的基本思路；
- (4) 认识到系统测试的重要性，很多问题都是在实际演示过程中逐步暴露并修正的；
- (5) 提高了文档整理能力，能够按照课程设计模板对项目进行系统性总结。

当然，系统仍有可以继续完善的地方。例如：

- (1) 可以增加统计报表，如自习室利用率、学生违约次数排行等；
- (2) 可以增加更细粒度的封禁策略，如累计多次违约自动延长封禁时间；
- (3) 可以进一步美化前端界面，并加入更多操作提示；
- (4) 可以将密码改为加密存储，以提高安全性；
- (5) 可以增加单元测试和接口自动化测试，提升系统工程化程度。

总体来看，本次课程设计较好地完成了“数据库原理及应用课程设计”的目标。系统实现了较完整的业务流程，数据库结构较清晰，界面交互较直观，具备课程设计可交付的基本质量。

6、参考文献

- [1] 王珊, 萨师煊. 数据库系统概论 (第六版) [M]. 北京: 高等教育出版社, 2024.
- [2] 张海藩, 牟永敏. 软件工程导论 (第七版) [M]. 北京: 清华大学出版社, 2020.
- [3] 郑阿奇. MySQL 实用教程 (第四版) [M]. 北京: 电子工业出版社, 2022.
- [4] 李明, 王强. 基于 Spring Boot 和 Vue 的座位预约系统设计与实现[J]. 计算机时代, 2023(5): 56-59.
- [5] 张伟. 基于 B/S 架构的高校自习室管理系统设计[J]. 软件导刊, 2022, 21(3): 112-115.
- [6] 刘洋, 陈静. MyBatis-Plus 在持久层开发中的应用与优化[J]. 电脑编程技巧与维护, 2023(8): 45-47.
- [7] 赵敏. 结合 Vue.js 框架的前后端分离架构研究[J]. 信息技术与信息化, 2021(10): 88-90.
- [8] 孙超. 关系型数据库 MySQL 在并发控制中的应用探析[J]. 数据库技术, 2022(4): 33-36.
- [9] 周杰. 基于微服务架构的校园资源管理平台的研究与实现[J]. 计算机工程与科学, 2024, 46(1): 102-108.
- [10] 吴迪. 前后端分离模式下的 Web 系统安全性研究[J]. 网络安全技术与应用, 2023(2): 77-79.
- [11] Smith J, Johnson A. Design and Implementation of a Campus Resource Reservation System Based on Spring Boot[C]// 2023 International Conference on Computer Science and Information Technology. IEEE, 2023: 120-125.
- [12] Wang L, Chen Y. Research on Web Front-end Development Based on Vue.js[J]. Journal of Physics: Conference Series, 2021, 1827(1): 012015.
- [13] Garcia M. Optimizing Database Transaction Management in High-Concurrency Scenarios[J]. IEEE Transactions on Knowledge and Data Engineering, 2022, 34(6): 2850-2862.
- [14] Davis R, Evans K. A comparative study of modern ORM frameworks in Java: MyBatis vs Hibernate[J]. Software: Practice and Experience, 2023, 53(4): 980-995.
- [15] Lee C H. Architecture Design of B/S Application System based on Separation of Front and Back Ends[C]// 2022 4th International Conference on Software Engineering and Application. IEEE, 2022: 45-50.